Document number	P0401R0
Date	2015-07-08
Project	Programming Language C++, Library Evolution Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

Extensions to the Allocator interface

Introduction

Despite all the changes to allocators in each revision of the C++ standard, they still have a very limited interface for their main function: allocating memory.

This proposal offers some ideas for new `allocate()` overloads to help specific use cases.

Getting over allocation

Many allocators will only allocate fixed-size chunks of memory, rounding up requests for smaller sizes to the nearest chunk size. Because the Allocator interface doesn't provide any means for an allocator to expose this information to code using it, this can lead to very inefficient memory usage. For example a `vector` that allocates 36 bytes might result in a 64-byte chunk being allocated, but the vector only uses the 36 bytes it asked for, wasting 43% of the memory. If the vector then grows such that it needs 40 bytes it will ask the allocator for it and be given a *second* 64-byte chunk, even though the existing one was large enough already.

This could be addressed by adding a new optional overload of `allocate` that returns the size that was actually allocated e.g.

```
pointer allocate(size_type n, size_type& result, const_void_pointer hint = {});
```

This would allow allocators to inform the container when they have over-allocated and for containers to use the additional capacity. `std::vector` and `std::string` would use the `result` parameter as their new capacity and so could avoid a re-allocation if the size grows beyond the originaly requested capacity but would still fit in the actually-allocated capacity.

Interaction with N4524

The proposal in Respect vector::reserve(request) Relative to Reallocation (N4524) would not forbid `std::vector::reserve(size_type)` from taking advantage of this new interface. All that would be required is for the allocator to guarantee that it won't over-allocate optimistically, i.e. it will only return a larger size than requested when doing so is an unavoidable consequence of how it obtains memory. If the allocator is incapable of reserving fewer than N bytes when requested to allocate n (where N >= n) then it can't meet the "guarantee" that N4524 asks for anyway. Informing the `vector` of that additional amount has no downside (except possibly users being confused).

So to meet the aim of N4524, `vector::reserve()` could guarantee to only *ask* for exactly that number of bytes, and `std::allocator` could guarantee not to over-allocate on the off-chance it might be used (which still allows returning a larger size if that's what really gets allocated anyay). Custom allocators should be allowed to over-allocate as much as they like though. If users want a guarantee that `vector::reserve()` doesn't waste memory they should be sure to use an allocator that won't waste memory. That should provide a way to please everyone, because the decision of whether to over-allocate aggressively will be made by the allocator (where such decisions belong), and vector's behaviour can be customized by using a different allocator.

Would "realloc" be better?

An alternative that would address the same problem, at least partially, would be to add a `realloc`-like function as proposed by N3495. As proposed that is a very different API to `std::realloc` which is also responsible for "moving" data from the old storage

to the new, but that should be done by the container not the allocator. The N3495 proposal uses an exception of type `std::bad_alloc` to report when the existing memory region cannot be resized, which might be the common case for many allocators and for many allocations (e.g. even if the allocator supports the resize operation, doubling the size of a vector might require a new memory chunk anyway). That would also require containers to add extra code to try and resize, and then if that throws fall back to allocating and moving elements as done today. That adds exceptions and extra branching to the normal control-flow path. For fixed-size allocators it makes more sense, and is much simpler for containers to adapt to, if the allocator is able to over-allocate on the initial request and inform the caller how much memory was made available. Note, however, that my proposal doesn't help the shrink-to-fit case where resizing would be used to return unused memory (which not all allocators would be able to support anyway). It also doesn't help for allocators which don't over-allocate initially but might be able to grow the memory region without re-allocating (e.g. because the next page in the virtual address space is not mapped yet and so could be made available).

Implementation

The default implementation in `allocator_traits` would be trivial:

```
pointer
allocate(allocator_type& a, size_type n, size_type& result, const_void_pointer  hint = {})
{
  pointer p = allocate(a, n, hint);
  result = n;
  return p;
}
```

Whether `std::allocator` does anything different would be unspecified and depend on the implementation of `std::allocator`. An allocator that uses jemalloc could implement the function like so:

```
pointer
jemallocator<T>::allocate(size_type n, size_type& result, const void* = nullptr)
{
  void* ptr = mallocx(n * sizeof(T), MALLOCX_ALIGN(alignof(T)));
  result = sallocx(ptr, 0) / sizeof(T);
  return static_cast<T*>(ptr);
}
```

Experimental results

I modified the libstdc++ `vector` and `allocator_traits` to use this new overload, implemented the `jemallocator` function shown above, and ran this code in a loop:

```
  {
    std::vector<int, jemallocator<int>> v;
    v.resize(7);
    for (int i = 0; i < 1000; ++i)
      v.push_back(1);
  }
```

The test was run twice, once with `allocate` calling `sallocx` to obtain the actual size that was allocated, and again with the traditional behaviour (i.e. doing `result = n` instead so that extra space was not available to the vector).

When the allocator returned the actual size it was 15% faster and the final capacity was 1024 after eight allocations, compared to 1792 after nine allocations. (The difference was due to whether the first allocation returned `result=8` or `result=7`, and so whether subsequent doubling in size went up in powers of two until reaching 1024, or as multiples of 7, 14, ...).

Without the initial `v.resize(7)` the capacity in both tests grew as powers of two, and jemalloc always returned exactly what was requested, so there was no wasted space that the vector wasn't told about. However the test that returned the actual size was 3% slower (probably due to the overhead of calling `sallocx`). Clearly the benefits will depend on workload, but there is potential for significant speedup with a suitable allocator. An allocator which doesn't require a separate call to obtain the real size of the allocation might avoid that 3% overhead.

Other microbenchmarks showed no significant difference in performance with or without the `sallocx` call. More numbers would be useful.

Nothrow overload of allocate()

Although they're not standard C++ there are exception-free environments (i.e. where everything is compiled with an option like `-fno-exceptions`) that range from the smallest systems to the largest.

`shrink_to_fit()` is hard to use in such codebases, because it's supposed to be a non-binding request, but if re-allocating fails then

a naïve implementation will terminate rather than leaving the container unchanged. If containers obtained memory using `operator new` then `shrink_to_fit` could be made to use `new(std::nothrow)` when exceptions are disabled, but the Allocator API has no equivalent.

It's also hard to use allocators in custom containers which don't use exceptions for error handling, since allocators can only report failure via exceptions. Such containers can in theory catch `bad_alloc` and report an error some other way, but that's impossible if `-fno-exceptions` is used for compilation.

This could be addressed by adding a new optional overload of `allocate`:

```
pointer allocate(nothrow_t, size_type n, const_void_pointer hint = {}) noexcept;
```

and the corresponding function in `allocator_traits`, which would simply call `a.allocate(n, hint)` if the allocator doesn't support the overload, but on failure would swallow any exception and return a null pointer.

Implementations wishing to support exception-free codebases would ensure that `std::allocator` provides an implementation with an exception-free code path, and users could use their own allocators that define the overload.

Implementations that don't care about supporting exception-free environments don't need to do any extra work, just define the overload with the default behaviour. Avoiding exceptions would be a quality of implementation issue, not a conformance one.

For the same reasons, the over-allocating overload should also get a nothrow variant.

Proposed Wording

None at this time, the aim of this paper is only to gauge interest in these extensions, and consider the interaction with N4524.

Any new overloads are likely to involve changes to:

- `std::allocator`
- `std::allocator_traits`
- `std::memory_resource`
- `std::polymorphic_allocator`